

● MANIFEST V3 · LOCAL-FIRST · AI-POWERED

TECHNICAL DOCUMENTATION

Resume Scorer

AI-powered resume screening & recruitment automation
— from inbox to interview, end to end.

Chrome Extension

Node · Express Backend

Groq / Anthropic / Gemini

Local Embeddings + RAG

SQLite · sqlite-vec · Pinecone

Google Calendar / Meet / Gmail

VERSION

0.2.0

DATE

29 May 2026

STATUS

MVP — Active

CLASSIFICATION

Internal

Table of Contents

1	Executive Summary	— what the product is & why
2	Project Scope	— in-scope, out-of-scope, assumptions & constraints
3	Use Cases	— actors, use-case catalogue & key scenarios
4	System Design & Architecture	— tiers, components & data flows
5	Data Model	— database schema & storage strategy
6	API Reference	— backend HTTP endpoints
7	Technology Stack	— every layer, library & why
8	Security & Privacy	— the rules baked into the design
9	Setup & Deployment	— how to run it end to end
10	Appendix	— glossary & references

Executive Summary

Resume Scorer is a local-first recruitment assistant that turns a recruiter's email inbox into a scored, searchable, and actionable candidate pipeline. It pairs a **Chrome (Manifest V3) extension** with a small **Node.js / Express backend** so that resumes flowing into Outlook or Gmail are automatically extracted, scored by a large language model, indexed for semantic search, and pushed through configurable hiring automations — all without sending an API key to the browser.

2INBOXES (OUTLOOK +
GMAIL)**3**AI PROVIDERS,
SWAPPABLE**6**DASHBOARD ANALYTICS
CHARTS**0**API KEYS IN THE
BROWSER

The problem it solves

Recruiters receive resumes as email attachments and triage them by hand: open the PDF, skim it, guess a fit, copy details into a sheet, then chase scheduling over more email. Resume Scorer collapses that loop. The moment a resume lands, it is parsed, scored 0–100 with a category breakdown, enriched with structured candidate fields, made searchable in plain English, and — where the recruiter opts in — routed into automations that send assessment links, book interviews with live Google Meet links, or send a polite rejection.

What makes it different



Key-safe by design

Every AI call is brokered by the local backend. The extension never holds a provider key, and OAuth uses PKCE with no client secret.



Cheap to run

Local embeddings (Transformers.js), score caching, and a regex-first query parser keep token spend near zero on the free tier.



Semantic search + RAG chat

"Show me backend devs with 3+ years" is answered by a SQL filter and a vector index, then narrated by the LLM.



Visual automation

An N8N-style canvas wires triggers → filters → actions: send OA, schedule interviews, reject — with a full dry-run mode.

Project Scope

This section draws the boundary of the MVP: what the system does, what it deliberately does not, and the assumptions the design rests on.

2.1 In scope

CAPABILITY	DESCRIPTION
Inbox connection	OAuth login to Microsoft Outlook (Graph API) and Google Gmail, with automatic token refresh.
Attachment discovery	List recent emails that carry <code>.pdf</code> / <code>.docx</code> attachments; ignore inline images & signatures.
Text extraction	Server-side parsing of PDF (<code>pdf-parse</code>) and DOCX (<code>mammoth</code>) into raw text.
AI scoring	0–100 overall score plus a 5-axis breakdown (experience, skills, education, clarity, impact), summary, strengths, concerns, recommendations.
Candidate enrichment	Structured extraction of name, contact, current role/company, years of experience, education, skills, locations, remote history, certifications & more.
Classification	Auto-categorisation into role categories and a role title for each candidate.
Semantic search & RAG	Local embeddings indexed in sqlite-vec (and optionally Pinecone) with FTS5 keyword search for hybrid retrieval.
AI chat	Cross-resume mode ("find me...") and single-candidate mode ("tell me about this person"), with a regex pre-parser to save tokens.
Recruiter dashboard	Web UI at <code>localhost:8787</code> — KPIs, six charts, searchable candidate table, drag-drop upload, per-candidate modal.
Excel export	One-click <code>.xlsx</code> export of candidates with scores & summaries (<code>exceljs</code>).
Automation builder	Visual canvas: triggers, filters, branch/top-N logic, and actions (send OA, schedule interview, reject, notify, log) with dry-run.
Interview scheduling	Finds the first free slot across selected interviewers' Google Calendars and books an event with a fresh Google Meet link.

2.2 Out of scope (for this MVP)

- **OCR of scanned / image-only PDFs** — only text-bearing documents are parsed.
- **Multi-user accounts, roles & permissions** — single local operator.

- **Cloud / multi-tenant hosting** — backend runs on `localhost`.
- **Production hardening** — rate limiting, audit logs, secrets vaulting beyond `.env`.
- **Deep ATS / HRIS integrations** beyond Gmail / Calendar actions.
- **Scheduled / webhook triggers** — the only automation trigger is "Manual run".
- **Mobile or non-Chromium browsers** — targets Chrome / Edge (Chromium).
- **Bias auditing & compliance certification** of the AI scoring model.

2.3 Assumptions & constraints

OPERATING ASSUMPTIONS

- The operator runs the backend locally and keeps it running while using the extension.
- The operator can complete a one-time Azure (Microsoft) and Google Cloud OAuth app registration.
- An AI provider key (Groq is free) is available and placed in `backend/.env`.
- Resumes arrive as machine-readable PDF/DOCX files in the connected inbox.

HARD CONSTRAINTS (ARCHITECTURAL RULES)

- **No API key may ever live in the extension** — all model calls go through the backend.
- **Manifest V3 only** — background logic is a service worker, not a persistent page.
- **OAuth is PKCE with a public client** — no client secret is stored anywhere in the extension.
- **Text extraction & AI calls happen in the backend**, never in the browser.
- **Secrets stay out of git** — `.env`, token caches and `node_modules` are git-ignored.

Use Cases

3.1 Actors

ACTOR	ROLE IN THE SYSTEM
Recruiter / Operator	Primary human user. Connects inboxes, reviews scores, searches, chats, and runs automations.
Candidate	External party whose resume is scored and who receives OA links, interview invites or rejections.
Interviewer / Panel	Internal staff whose calendar availability drives interview scheduling.
AI Provider	External LLM (Groq / Anthropic / Gemini) that scores, enriches and narrates.
Identity Providers	Microsoft Identity & Google OAuth — authenticate inbox / calendar access.

3.2 Use-case catalogue

ID	USE CASE	OUTCOME
UC-1	Connect an inbox	Recruiter signs into Outlook and/or Gmail via OAuth; tokens stored & auto-refreshed.
UC-2	Score a single resume	Pick an email attachment → backend extracts text → LLM returns score + review rendered in popup.
UC-3	Bulk upload & score	Drag-drop one or many PDFs/DOCX into the dashboard; each is scored, enriched & indexed.
UC-4	Browse the pipeline	Overview KPIs + six charts; searchable candidate table with per-candidate detail modal.
UC-5	Semantic / filtered search	Plain-English queries ("backend devs, 3+ yrs, remote") resolved by SQL filter + vector search.
UC-6	AI chat about candidates	Ask across all resumes or about one candidate; grounded answers via RAG.
UC-7	Export shortlist	Download an <code>.xlsx</code> of candidates with scores & AI summaries.
UC-8	Build & run an automation	Wire trigger → filter → actions; preview with dry-run; execute & review run history.
UC-9	Schedule an interview	For qualifying candidates, find a free panel slot & book a Calendar event with a Meet link.

ID	USE CASE	OUTCOME
UC-10	Send OA link / rejection	Templated Gmail send with <code>{{placeholders}}</code> ; warm / direct / encouraging tones.

3.3 Key scenario — Score a resume (UC-2)

1 Recruiter opens the extension & selects an email

The side-panel popup lists recent messages that carry PDF/DOCX attachments, fetched via Microsoft Graph / Gmail API using the stored access token.

2 Extension downloads the raw attachment

The attachment is fetched as base64 (`contentBytes`) — no parsing happens in the browser.

3 Extension POSTs to the local backend

`POST /score { filename, contentType, contentBase64 }` is sent to `http://localhost:8787`.

4 Backend extracts text & calls the LLM

`extract.js` decodes & parses the document; `ai.js` sends a strict-JSON prompt to the configured provider.

5 Backend persists, indexes & returns the result

The score, review & enriched fields are written to SQLite; chunks are embedded into the vector index; JSON is returned to the popup and rendered as a score + breakdown chart.

EDGE CASES THE DESIGN HANDLES

- **Duplicate scoring** — a unique `(email_id, attachment_id)` index makes re-scoring the same attachment a no-op.
- **Scanned PDFs** — if too little text is extracted, the UI fails loudly ("could not extract enough text").
- **Candidates with no email** — automation marks them `skipped`, not failed.
- **Dry-run** — every email & booking can be previewed (exact subject/body/attendees) before anything is sent.
- **Graph cycles** — the automation engine refuses to run a workflow wired back on itself.

System Design & Architecture

A three-tier, local-first architecture: a browser client, identity providers, and a local backend that brokers every privileged operation — AI calls, document parsing, persistence and third-party automation.

4.1 High-level architecture

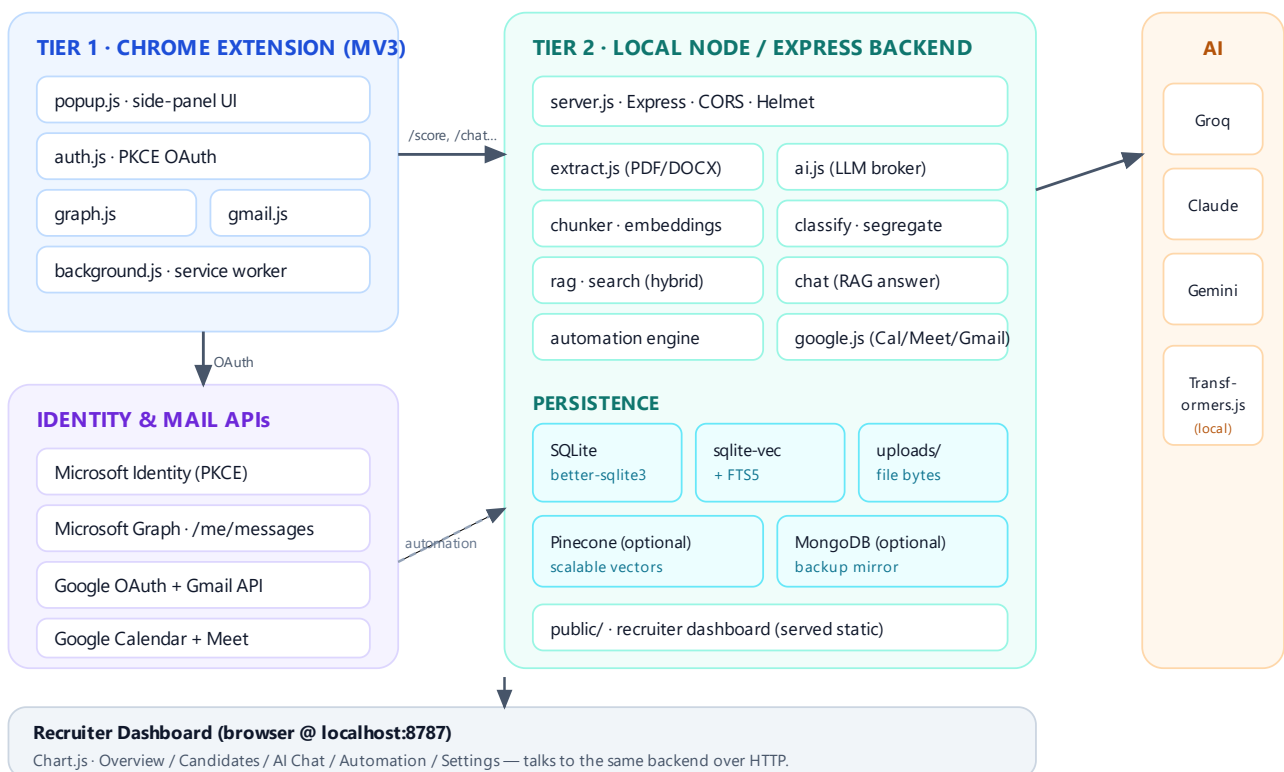


Figure 1 — Component & tier architecture. The backend is the single trust boundary for keys, parsing and AI.

4.2 Component responsibilities

COMPONENT	TIER	RESPONSIBILITY
<code>popup.js</code>	Extension	Side-panel UI; orchestrates login, listing, scoring & rendering.
<code>auth.js</code>	Extension	PKCE OAuth: code_verifier/challenge, token exchange, storage & refresh.
<code>graph.js</code> / <code>gmail.js</code>	Extension	List messages & download attachments from Outlook / Gmail.

COMPONENT	TIER	RESPONSIBILITY
<code>background.js</code>	Extension	MV3 service worker — alarms, redirect-URI logging, lifecycle.
<code>server.js</code>	Backend	Express app; all HTTP routes, CORS, Helmet, static dashboard.
<code>extract.js</code>	Backend	Decode base64 → text via pdf-parse (PDF) / mammoth (DOCX).
<code>ai.js</code>	Backend	Provider-agnostic LLM broker (Groq / Anthropic / Gemini); strict-JSON parsing.
<code>chunker.js</code> · <code>embeddings.js</code>	Backend	Split resume text into chunks; embed locally or via Google/Voyage.
<code>classify.js</code> · <code>segregate.js</code>	Backend	Assign role category / title; structured candidate field extraction.
<code>rag.js</code> · <code>search.js</code>	Backend	Hybrid retrieval (vector + FTS5 BM25) for search & chat grounding.
<code>chat.js</code>	Backend	RAG answer generation; regex pre-parser for filter-style queries.
<code>db.js</code>	Backend	SQLite schema, migrations, vec0 & FTS5 virtual tables.
<code>automation.js</code> · <code>automationDb.js</code>	Backend	Workflow engine (nodes, edges, run history) & its persistence.
<code>google.js</code>	Backend	Calendar free/busy, event + Meet creation, Gmail send.
<code>pinecone.js</code> · <code>mongo.js</code>	Backend	Optional scalable vector store & backup mirror with graceful fallback.

4.3 Data flow — RAG chat & semantic search

1 Query pre-parse

A regex layer extracts structured intent (category, years, skills, location, top-N) so deterministic filters run in SQL *before* any LLM call — saving tokens.

2 Hybrid retrieval

The query is embedded and matched against `chunk_vectors` (cosine) while FTS5 BM25 retrieves keyword hits; results are merged & ranked.

3 Context assembly

Top chunks plus structured candidate rows are packed into a compact grounding context.

4 LLM narration

The model writes the final natural-language answer (optionally streamed via `/chat/stream`), grounded only in retrieved evidence.

4.4 Data flow — automation run

1 Trigger

Operator presses *Execute* on a workflow (Manual-run trigger).

2 Filter & branch

Candidates flow through filters (score $\geq$ X, category, years, skills) and branch / top-N logic into action streams.

3 Actions

Send OA (Gmail), schedule interview (Calendar free/busy $\rightarrow$ event + Meet), polite rejection, notify, log — each paced to respect rate limits.

4 Run history

Every per-candidate result (sent / skipped / failed, with the API's error message) is recorded for audit. Dry-run renders everything without sending.

Data Model

SQLite (via `better-sqlite3`) is the single source of truth, extended with `sqlite-vec` for vector search and FTS5 for keyword search. Pinecone and MongoDB are optional mirrors enabled by configuration.

5.1 Core tables

TABLE	PURPOSE & NOTABLE COLUMNS
<code>resumes</code>	One row per scored candidate. Holds <code>score</code> , <code>category</code> , <code>role_title</code> , <code>review_json</code> , <code>raw_text</code> , plus enriched fields: <code>candidate_name</code> , <code>email</code> , <code>phone</code> , <code>location</code> , <code>years_experience</code> , <code>current_title/company</code> , <code>top_skills</code> , <code>education_json</code> , <code>certifications</code> , <code>companies</code> , <code>domains</code> , <code>remote_worked</code> , <code>managed_people</code> , and more. A unique partial index on <code>(email_id, attachment_id)</code> de-dupes re-scores.
<code>chunks</code>	Resume text split into retrieval-sized chunks; FK to <code>resumes</code> with cascade delete.
<code>chunk_vectors</code>	<code>vec0</code> virtual table — fixed-dimension embeddings (<code>EMBED_DIM</code> ; default 768) for cosine search.
<code>chunks_fts</code>	FTS5 virtual table mirroring chunk text via triggers — BM25 / keyword retrieval for hybrid search.
<code>chat_threads</code>	Conversations; optionally bound to a single <code>resume_id</code> for per-candidate chat.
<code>chat_messages</code>	Per-thread message log (role + content), indexed by <code>(thread_id, id)</code> .
<i>automation tables</i>	Workflows (nodes/edges), run history, interviewers & availability windows, and message templates.

5.2 Storage strategy

STORE	ROLE	BEHAVIOUR
SQLite + <code>sqlite-vec</code>	Primary	Always on. Holds all relational data, vectors & FTS index in one local file (<code>data.db</code>).
Pinecone	Optional (scale)	When <code>PINECONE_API_KEY</code> is set, vectors dual-write to Pinecone + <code>sqlite-vec</code> ; reads prefer Pinecone and fall back to <code>sqlite-vec</code> on any error.
MongoDB	Optional (mirror)	Backfill / backup mirror of resume records for durability.
uploads/	Blob store	Original PDF/DOCX bytes on disk; <code>file_path</code> references them for re-download / preview.

GRACEFUL DEGRADATION

A Pinecone outage degrades to local sqlite-vec search rather than returning empty results — availability is never gated on an external vector store.

SECTION 6

API Reference

All routes are served by the local Express backend at <http://localhost:8787>. Methods are colour-coded below.

GET read **POST** create / action **PUT** update **DEL** delete

6.1 Scoring & resumes

METHOD	PATH	PURPOSE
POST	/score	Score an attachment (filename, contentType, contentBase64).
POST	/score-text	Score pre-extracted raw text.
POST	/classify	Categorise a resume / assign role title.
POST	/summarize	Return (cached) summary / strengths / concerns.
GET	/resumes	List candidates (filterable).
GET	/resumes/by-category	Candidates grouped by category.
GET / POST	/resumes/search	Search candidates (query / filters).
GET	/resumes/:id · /:id/file	Get one candidate / download original file.
GET	/resumes/export.xlsx	Excel export of candidates + summaries.
DEL	/resumes/:id	Delete a candidate (cascades chunks/vectors).
GET	/stats	Aggregate metrics powering dashboard charts.

6.2 Search, chat & segregation

METHOD	PATH	PURPOSE
POST	/parse-query	Regex pre-parse a natural-language query into filters.
POST	/chat · /chat/stream	RAG chat (buffered / streamed).
GET / DEL	/threads · /threads/:id/messages	Manage chat threads & history.
POST	/segregate · /segregate/extract	Extract & segregate structured candidate fields.

6.3 Automation, Google & Gmail

METHOD	PATH	PURPOSE
GET POST PUT DEL	/automation/workflows[:id]	CRUD on automation workflows.
POST	/automation/workflows/:id/run	Execute a workflow (supports dry-run).
GET	/automation/runs[:id]	Run history & per-candidate results.
GET	/automation/google/auth · /callback · /status	Google OAuth connect flow & status.
GET POST DEL	/automation/interviewers[:id] · /availability	Manage interviewers & availability windows.
GET POST PUT DEL	/automation/templates[:id]	Message templates with placeholders.
GET	/gmail/status · /gmail/auth · /gmail/messages	Gmail connection, listing & attachments.
GET	/health · /mongo/status	Backend & storage health probes.

Endpoint list reflects the routes registered in `server.js`. Request/response shapes follow the JSON contract described in §4 (e.g. the `/score` result returns `score`, `breakdown`, `summary`, `strengths`, `concerns`, `recommendations`).

Technology Stack

Every layer favours "works end-to-end, cheap to run, no build step." Plain modern JavaScript throughout — no bundler, no framework for the MVP.

7.1 Client — Chrome Extension

TECHNOLOGY	ROLE
Manifest V3	Extension platform; service-worker background, side-panel UI.
Vanilla JS (ES modules)	No framework / build step — loadable unpacked directly.
Chrome APIs	<code>identity</code> (launchWebAuthFlow), <code>storage</code> , <code>alarms</code> , <code>sidePanel</code> .
Web Crypto (SHA-256)	PKCE code-challenge generation in-browser.

7.2 Backend — Node.js service

TECHNOLOGY	ROLE
Node.js 22 + Express 4	HTTP server & routing (ES modules).
Helmet · CORS	Baseline security headers & cross-origin access for the extension.
pdf-parse · mammoth	PDF & DOCX text extraction.
better-sqlite3	Synchronous, fast embedded SQLite driver.
sqlite-vec	Vector search inside SQLite (<code>vec0</code> virtual tables).
exceljs	Excel (<code>.xlsx</code>) export of candidate data.
dotenv	Loads secrets from git-ignored <code>.env</code> .

7.3 AI & retrieval

TECHNOLOGY	ROLE
Groq — Llama 3.3 70B	Default scoring/chat LLM (free tier, fast).
Anthropic Claude	Alternative provider (<code>AI_PROVIDER=anthropic</code>).
Google Gemini	Alternative provider (<code>AI_PROVIDER=gemini</code>).
Transformers.js (@huggingface/transformers)	Local, key-free embeddings (default <code>all-MiniLM-L6-v2</code> , 384-d).

TECHNOLOGY	ROLE
Google text-embedding-004 / Voyage	Optional hosted embedding providers (768-d / 512-d).
Pinecone	Optional managed vector DB for scale + fault tolerance.
MongoDB	Optional record mirror / backup.

7.4 Dashboard & integrations

TECHNOLOGY	ROLE
Chart.js 4.4	Six analytics charts on the recruiter dashboard.
HTML / CSS (Manrope)	Static dashboard served from <code>backend/public</code> .
Microsoft Graph API	List Outlook mail & download attachments.
Gmail API	List Gmail mail, download attachments & send messages.
Google Calendar API + Meet	Free/busy lookup, event creation, auto Meet links (<code>conferenceData</code>).
Microsoft Identity / Google OAuth	PKCE (extension) & web OAuth (backend) for delegated access.

Security & Privacy



No keys in the browser

Provider keys live only in `backend/.env`. Extension code is fully readable, so it never holds a secret.



PKCE, no client secret

Auth-code + PKCE via a public client. SPA redirect URI enables CORS token exchange directly.



Local-first data

Resumes, scores & chats stay on the operator's machine in SQLite. No cloud account required.



Secrets out of git

`.gitignore` covers `.env`, token caches, `node_modules` & the database.

8.1 Token handling

- Access / refresh tokens are stored in `chrome.storage.local` (extension) and refreshed ~60 s before expiry.
- Backend-side Google OAuth tokens are obtained via the web flow and used only for the operator's own Calendar/Gmail actions.
- Least-privilege scopes: Microsoft `User.Read` + `Mail.Read`; Google `gmail.send`, `calendar.events`, `calendar.readonly`.

8.2 Data minimisation & failure behaviour

- Only attachments ending in `.pdf` / `.docx` are processed; inline images & signatures are skipped.
- The UI fails loudly — login errors, empty inboxes and backend-down are surfaced, never silent.
- Automation defaults to safety: dry-run preview, candidate-without-email → `skipped`, partial failures recorded with the upstream error.

MVP CAVEAT

This is an MVP, not a hardened production system. Before any multi-user or internet-exposed deployment, add authentication on the backend, rate limiting, encrypted secret storage, and an AI-bias / fairness review of the scoring prompts.

Setup & Deployment

Three one-time manual steps, then run two processes. Full step-by-step lives in the project `README.md`; this is the operational summary.

9.1 Prerequisites

- Node.js 18+ (developed on Node 22) and a Chromium browser (Chrome / Edge).
- An Azure app registration (Microsoft) and a Google Cloud OAuth client for automation.
- An AI provider key — Groq is free (console.groq.com/keys).

9.2 Run the backend

#	ACTION
1	<code>cd backend && npm install</code>
2	<code>copy .env.example .env</code> — fill <code>GROQ_API_KEY</code> (and Google values for automation).
3	<code>npm run dev</code> → backend listens on <code>http://localhost:8787</code> .
4	Open <code>http://localhost:8787</code> for the recruiter dashboard.

9.3 Load the extension

#	ACTION
1	<code>chrome://extensions</code> → enable Developer mode → Load unpacked → select <code>extension/</code> .
2	Open the service-worker console, copy the printed redirect URI (<code>https://<id>.chromiumapp.org/</code>).
3	In Azure, add it as a Single-page application redirect URI; copy the Application (client) ID .
4	Paste the client id into <code>extension/config.js</code> ; reload the extension.
5	Click the toolbar icon → Connect Outlook (and/or Gmail) → start scoring.

DEFINITION OF DONE

`npm run dev` starts the backend; loading `extension/` unpacked shows a popup with "Connect Outlook"; after login the popup lists resume emails; clicking one renders a score + review; the dashboard shows the full pipeline.

9.4 Smoke tests

Automation logic is covered by `node test-automation.js` (29 tests — schema, CRUD, filters, template rendering, end-to-end dry-run, cycle detection, route handlers; no network calls).

Appendix

10.1 Glossary

TERM	MEANING
PKCE	Proof Key for Code Exchange — OAuth extension that secures the auth-code flow without a client secret.
MV3	Manifest V3 — current Chrome extension platform; background logic runs as a service worker.
RAG	Retrieval-Augmented Generation — ground the LLM in retrieved chunks before it answers.
Embedding	Numeric vector representing text meaning; nearby vectors = similar meaning.
sqlite-vec / vec0	SQLite extension & virtual-table type for storing & searching embeddings.
FTS5	SQLite full-text search module providing BM25 keyword ranking.
Hybrid search	Combining vector similarity with keyword (BM25) retrieval for better recall.
Dry-run	Execute an automation that renders all outputs without sending anything externally.
conferenceData	Google Calendar field that, on event creation, mints a fresh Google Meet link.

10.2 Key files & references

PATH	WHAT IT IS
<code>extension/</code>	Chrome MV3 extension (popup, auth, graph, gmail, background).
<code>backend/server.js</code>	Express app & all HTTP routes.
<code>backend/.env.example</code>	Full configuration reference (providers, embeddings, Pinecone, Google).
<code>backend/public/</code>	Recruiter dashboard (index.html, dashboard.js, automation.js, Chart.js).
<code>README.md</code>	Complete setup, Azure / Google registration & troubleshooting guide.
<code>CLAUDE.md</code>	Project context & architectural rules.